Unified Memory Frameworks

Updated 25 March 2026

> Unified memory frameworks are infrastructures that fuse distinct memory types into a single logical space for streamlined data movement and management.
>
> They employ techniques such as predictive migration and compiler-guided scheduling to enhance performance and reduce latency in heterogeneous environments.
>
> They play a key role in deep learning, HPC, and agentic systems by providing scalable, efficient, and coherent memory management across diverse hardware.

Unified memory frameworks encompass architectural, algorithmic, and software innovations that present abstraction layers or concrete mechanisms fusing heterogeneous or distributed memory spaces (e.g., CPU, GPU, PIM, high-speed flash, agent-specific memory banks) into a logically unified addressable or operational system. Such frameworks facilitate transparent or optimized data movement, sharing, and management while supporting diverse goals: scalability for large compute workloads, agentic memory for LLM-based systems, or robustness and coordination in multi-agent scenarios.

1. Foundational Concepts and Typologies

Unified memory refers to a principle or infrastructure in which distinct physical memory components—such as CPU DRAM, GPU HBM, PIM banks, or persistent storage—are composed into a single logical or virtual address space, or accessed via a unified operational API. Frameworks differ substantially in their granularity, from system-level virtual memory integration across accelerators to agent-level abstractions for experience extraction and storage. At the system and architecture level, unified memory systems may involve hardware-level page table integration, cache-coherent high-bandwidth links, and runtime memory management abstractions. In the agentic and algorithmic

context, unified memory frameworks blend persistent and working memory, often with tightly coupled extraction, retrieval, and management logic.

Unifying features include:

Single address space or logical namespace for application/programming ease (eliminates explicit device-host memory management).

Transparent or optimized migration: data placement, migration, and access pattern adaptation are orchestrated by schedulers, compiler-inserted hints, or policy engines.

Support for heterogeneity: works across device types, agent roles, or memory speeds.

2. Systems and Accelerator-Level Unified Memory

2.1 CPU-GPU Unified Memory Architectures

Contemporary CPU-GPU platforms such as NVIDIA Grace-Hopper implement unified memory via a system-wide page table accessed by both CPU and GPU, hardware-accelerated page migration driven by cache-coherent NVLink-C2C, and automatic or programmable migration policy (Schieffer et al., 2024, Li et al., 2024, Li, 2024). Key architectural details:

System memory (malloc/new) and managed memory (cudaMallocManaged) both mapped in one global virtual address space.

Hardware Address Translation Service (ATS) enables GPU TLB misses to be serviced via the CPU's system MMU, eliminating page-fault handler round-trips.

Migration is triggered by hardware access counters with tunable thresholds, allowing fine-grained control (4 KB/64 KB), unlike legacy UVM's 2 MB granularity.

Device First-Use (DFU) strategies inspired by OpenMP first-touch policies amortize page migration, especially in scientific workloads with repeated BLAS usage (Li, 2024).

This architecture enables zero-copy data movement for supported memory regions and fully cache-coherent direct access by either the GPU or CPU, while preserving NUMA characteristics and deeply impacting data locality-sensitive codes.

2.2 Unified PIM–Accelerator Memory

The IANUS architecture demonstrates a unified main memory system where processing-in-memory (PIM) resources and neural processing units (NPUs) share a single DRAM pool (Seo et al., 2024). Here, the PIM memory serves as both the direct compute substrate for PIM operations and as main off-chip memory for the NPU. PAS (PIM Access Scheduling) ensures mutual exclusion at the cycle level for conflicting accesses and dynamically maps workloads (matrix multiplication, attention, FC layers) to the most efficient unit given input characteristics and bandwidth models. This eliminates data duplication and enables an efficient partitioning of resources and interleaved operation scheduling.

Comparative Table: Key System-Level Frameworks

Framework/Arch.	Address Space Unification	Memory Migration Policy
Grace-Hopper UMA (Schieffer et al., 2024, Li et al., 2024, Li, 2024)	System-wide, via CPU MMU, NVLink-C2C	Counter-based or DFU, 4/64 KB pages
G10 (Zhang et al., 2023)	GPU, Host, Flash tier, extended page table	Compiler-inserted, bandwidth-aware
IANUS (Seo et al., 2024)	Unified PIM+NPU, all-to-all NoC	PAS, macro-commands, adaptive mapping
GPUVM (Nazaraliyev et al., 2024)	GPU-driven, via in-GPU runtime	On-demand paging over RDMA (GPU/NIC)

3. Unified Memory Scheduling and Management Algorithms

Robust performance under device oversubscription, heterogeneous access patterns, or high concurrency depends critically on management and scheduling mechanisms.

Predictive page migration: Transformers and pattern classifiers predict upcoming access deltas to guide prefetching and pre-eviction, reducing thrashing by up to 64.4 % (Long et al., 2022). Unified policy engines combine prediction frequency tables (maintained per address block) and live page chains (partitioned by recency) used for both prefetch and eviction actions, with frequency-based hot–cold ranking.

Compiler-directed scheduling: G10 (Zhang et al., 2023) leverages offline compiler and profile-driven analysis to predict tensor lifetimes/inactivity, allowing explicit insertion of `g10_pre_evict` , `g10_prefetch` system calls, optimizing cost–benefit trade-offs for migration (based on per-tensor size and bandwidth).

Automatic offload policies: Tools such as SCILIB-Accel use dynamic binary instrumentation to intercept BLAS routines, and combine device first-use page mapping with empirical thresholds to minimize data transfer overhead (Li, 2024).

4. Unified Memory in LLM-Based and Agentic Systems

4.1 Joint Long- and Short-Term Memory Abstractions

Frameworks such as Agentic Memory (AgeMem) (Yu et al., 5 Jan 2026) implement unified memory for LLM agents by treating both long-term (persistent memory bank) and short-term (current prompt context) memory as part of a single policy's state. Explicit tool-based actions (add, retrieve, update, delete, filter, summarize) are part of the agent action space, enabling end-to-end optimization via reinforcement learning with group-relative policy objectives (GRPO), allowing long-horizon credit assignment across memory operations and generation steps.

4.2 Extraction and Management Co-Optimization

UMEM (Ye et al., 11 Feb 2026) extends unified memory to agentic settings by coupling memory extraction (distilling insights from experience) and management (updating the memory bank) within a self-evolving framework. Semantic Neighborhood Modeling ensures that memory entries generalize beyond instance-specific noise, optimizing a marginal utility reward averaged over neighborhoods of semantically related queries, and employing GRPO to provide stable updates without a value network.

4.3 Modular Abstraction Libraries

MemEngine (Zhang et al., 4 May 2025) operationalizes a unified memory slot paradigm over LLM agents, supporting diverse memory organization and retrieval models (full, short-term, long-term, generative, bank, hierarchical), with plug-in storage backends and retrievers. The framework separates core memory abstractions (slots, embeddings, context) from per-model operations, supporting modular extensibility and interoperability across tools (LangChain, AutoGPT).

Comparative Table: LLM–Agent Memory Frameworks

Framework	Unified Memory Model	Key Memory Operations
AgeMem (Yu et al., 5 Jan 2026)	STM+LTM in agent state, tool API	Add, Retrieve, Filter, Summarize

Framework	Unified Memory Model	Key Memory Operations
UMEM (Ye et al., 11 Feb 2026)	Joint extraction+management	Add, Update
MemEngine (Zhang et al., 4 May 2025)	Slot abstraction, unified API	Store, Recall, Manage

5. Unified Operation Languages and Formal Specifications

Text2Mem (Wang et al., 14 Sep 2025) advocates for a schema-driven, formally specified memory operation language. This framework defines a core set of expressive, mutually exclusive operations (encode, update, promote, demote, split, lock, expire, retrieve, summarize, etc.), each as a JSON schema instance with semantic invariants (e.g., locked items can't be deleted, expire requires a TTL), parser-based NL→operation mapping, validators, adapters to standard memory backends (SQL prototypes, MemGPT, etc.), and a unified execution contract for result reporting. Such formalization enables cross-system safety, determinism, and portability, as well as throughput- and correctness-oriented benchmarking via Text2Mem Bench.

6. Application Domains and Impact

Unified memory frameworks have demonstrable impact across several domains:

Deep learning scaling: Transparent tiered migration and unified address spaces allow large-model training beyond GPU DRAM, with >90% of ideal throughput and up to 1.75× speedup over contemporary schemes (Zhang et al., 2023).

High-performance computing and scientific applications: Automatic, transparent BLAS offloading drastically reduces code porting effort and amortizes data migration cost, yielding 1.9–3.0× speedup in quantum chemistry/physics codes (Li et al., 2024, Li, 2024).

LLM Agents: Unified agentic memory models allow RL-driven, end-to-end optimization yielding higher memory quality, improved reasoning accuracy, and reduced prompt context redundancy (Yu et al., 5 Jan 2026, Ye et al., 11 Feb 2026).

Multi-Robot Collaboration: RoboOS-NeXT (Tan et al., 30 Oct 2025) establishes a shared Spatio-Temporal-Embodiment Memory (STEM), enabling lifelong adaptivity, robust failure recovery, and scalable parallelism in multi-robot teams.

Dialog and Knowledge Memory: The unified framework for dialog memory (Hu et al., 3 Jan 2026) shows that key representation, maintenance, and

retrieval policies—more than dependence on particular graph vs. flat memory architecture—dominate performance.

7. Limitations, Open Problems, and Future Directions

Unified memory frameworks, while addressing many scalability and usability barriers, face open challenges:

Policy generalization: Current predictive migration/classification models depend on patterns seen during profiling; dynamically emerging access signatures pose adaptation challenges (Long et al., 2022, Zhang et al., 2023).

Hardware–software co-design: Efficient silicon implementation of model-driven schedulers, as well as fine-grained page and buffer alignment, is still a moving target (Li, 2024).

Operational extensions: Richer memory operations (merging, snapshotting, structured summarization), learned or adaptive schemas, and cross-agent or multi-device coherency remain active areas (as in Text2Mem (Wang et al., 14 Sep 2025), RoboOS-NeXT (Tan et al., 30 Oct 2025)).

Compositionality and abstraction: Unifying across sequence, graph, and hierarchy at both system and model levels is under ongoing study, notably in LLM memory (Zhang et al., 4 May 2025), dialog frameworks (Hu et al., 3 Jan 2026), and world models (Dong et al., 9 Oct 2025).

Emerging trends include on-hardware AI accelerators for memory management (Long et al., 2022), multi-tier plug-and-play memory backends, ongoing formalization of cross-platform memory operation languages, and increasing generalization of agentic memory via semantic and group-level objectives.

In summary, unified memory frameworks—spanning compute systems, memory management algorithms, agentic memory, and operation languages—support scalable, efficient, and portable compute and intelligence across contemporary architectures and agentic platforms. They integrate heterogeneous memory tiers or cognitive memory modalities, leveraging predictive migration, learned scheduling, and formal operation interfaces to bridge the gap between hardware, algorithms, and agentic memory-centric AI (Schieffer et al., 2024, Seo et al., 2024, Zhang et al., 2023, Yu et al., 5 Jan 2026, Ye et al., 11 Feb 2026, Dong et al., 9 Oct 2025, Wang et al., 14 Sep 2025).

Markdown Report Issue Upgrade to Chat

References (14)

1. Harnessing Integrated CPU-GPU System Memory for HPC: a first look into Grace Hopper (2024)
2. Automatic BLAS Offloading on Unified Memory Architecture: A Study on NVIDIA Grace-Hopper (2024)
3. Performant Automatic BLAS Offloading on Unified Memory Architecture with OpenMP First-Touch Style Data Movement (2024)
4. IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System (2024)
5. G10: Enabling An Efficient Unified GPU Memory and Storage Architecture with Smart Tensor Migrations (2023)
6. GPUVM: GPU-driven Unified Virtual Memory (2024)
7. An Intelligent Framework for Oversubscription Management in CPU-GPU Unified Memory (2022)
8. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents (2026)
9. UMEM: Unified Memory Extraction and Management Framework for Generalizable Memory (2026)
10. MemEngine: A Unified and Modular Library for Developing Advanced Memory of LLM-based Agents (2025)
11. Text2Mem: A Unified Memory Operation Language for Memory Operating System (2025)
12. RoboOS-NeXT: A Unified Memory-based Framework for Lifelong, Scalable, and Robust Multi-Robot Collaboration (2025)
13. Does Memory Need Graphs? A Unified Framework and Empirical Analysis for Long-Term Dialog Memory (2026)
14. Unified World Models: Memory-Augmented Planning and Foresight for Visual Navigation (2025)

Topic to Video (Beta)

No one has generated a video about this topic yet.

Upgrade to Generate Videos

All Videos

Subscribe on YouTube

Whiteboard

No one has generated a whiteboard explanation for this topic yet.

Upgrade to Generate Whiteboards

Follow Topic

Get notified by email when new papers are published related to **Unified Memory Frameworks**.

Follow Topic by Email

Continue Learning

1. How do unified memory frameworks streamline data migration across heterogeneous memory systems?
2. What role do hardware-level page table integration and cache-coherence play in unified memory architectures?
3. How does predictive page migration contribute to performance improvements in unified memory systems?
4. What are the main challenges in designing and implementing unified memory frameworks for large-scale applications?
5. Find recent papers about unified memory frameworks.

Related Topics

1. Unified Memory Mechanism
2. Unified Memory Pool Overview
3. Unified GPU Memory Pool Overview
4. Agentic Memory Framework Overview
5. Multi-Component Memory Architecture
6. Unified Memory Architecture (UMA) Overview
7. Agentic Memory Systems
8. Memory-Augmented LLM Agents

9. Memory-Augmented Reasoning
0. Persistent LLM Memory Systems

Refresh